BioEval: An LLM-Driven Framework for Evaluating Dataset Transparency, Reproducibility, and Information-Theoretic Rigor in Computational Simulation Papers

Joel Dietz (ORCID 0009-0007-4948-9192)

June 7, 2026

Abstract

Computational and agent-based simulation papers increasingly drive claims about biological coordination — ant and termite stigmergy, honeybee colony dynamics, firefly synchronization, and ant-colony optimization — yet their reproducibility is rarely assessed systematically. Two failures recur: datasets and code are declared without resolvable, verifiable provenance, and systems whose entire subject is information flow are described mechanistically without ever quantifying that information. BioEval is an LLM-driven evaluation framework that scores a paper on a versioned seven-dimension rubric. Six dimensions measure conformance to good data and reproducibility practice — data disclosure, dataset resolvability, code availability and versioning, code-to-data traceability, simulation derivation clarity, and reproducibility-package quality. A seventh, orthogonal dimension scores Information-Theoretic Rigor: whether the paper formalizes and quantifies the information content of the system it models (Shannon entropy, mutual and transfer entropy, channel capacity, communication bit rate). Scores are grounded in verified external evidence — resolved accessions and DOIs, live repository facts — rather than the paper's own claims. We apply BioEval (rubric v0.9.0) to a corpus of 14 insect-swarm and agent-based simulation projects. Transparency is uneven and reproducibility packaging is consistently weak; most strikingly, information-theoretic rigor is systematically low even though communication and coordination are the modeled phenomena. This deposition bundles the report together with the complete, MIT-licensed source code of the evaluation system.

Keywords: reproducibility · dataset transparency · information theory · agent-based simulation · stigmergy · swarm intelligence · code-to-data traceability · large language models · FAIR data

1 Introduction

Agent-based and computational simulations are now a primary vehicle for claims about biological coordination: how ant and termite colonies self-organize through stigmergy, how honeybee colonies allocate labor, how populations of fireflies synchronize, and how ant-colony optimization transfers these mechanisms to engineering. The credibility of such claims rests on two things that are easy to assert and hard to verify. First, the data and code behind a simulation must be disclosed in a way that a reader can actually resolve and re-run — declaring that a dataset 'is available' is not the same as providing a resolvable accession, a versioned repository, and a runnable package. Second, when the modeled system is itself a communication system, scientific rigor demands that the information it carries be formalized and quantified, not merely narrated.

BioEval addresses both gaps in a single instrument. It is an evaluation framework that submits a paper (by URL or PDF), extracts its real text, gathers external evidence, and uses a large language model to score the paper against a fixed, versioned rubric. The framework deliberately separates two orthogonal questions: does the work conform to good data and reproducibility practice, and — independently — does it bring information-theoretic rigor to a system whose very subject is information flow? This report explains how the system works and presents its evaluations of a corpus of insect-swarm and agent-based simulation projects.

2 The BioEval Framework

2.1 Pipeline

A submission is fetched through an SSRF-guarded path and its text is extracted from the real PDF (via a pure-JavaScript pdf.js build); if too little text can be recovered — for example a scanned, image-only document —

the evaluation errors out with a stated reason rather than scoring garbage. The extracted text is then analyzed by Anthropic's Claude in an evidence-gathering pass that resolves declared identifiers against Crossref and DataCite and inspects any linked code repository for live facts (license presence, releases, structure). Only after evidence is collected does the model assign the seven dimension scores together with a structured set of findings, gaps, and recommendations, all persisted and aggregated in a dashboard.

2.2 The seven dimensions

The rubric is versioned (currently v0.9.0, intentionally pre-1.0 while it is validated by reviewers) and stamped onto every evaluation so scores remain interpretable as the rubric evolves. Each dimension is scored 0–100 against fixed guideposts; the overall score is the weighted mean of all seven, using the weights below.

Dimension	Weight	What it tests
Data Disclosure	15%	Whether the paper names the data it uses and declares its provenance, access conditions, and licensing — not merely asserting that data exists.
Dataset Resolvability	12%	Whether declared datasets carry resolvable identifiers (accessions, DOIs, repository URLs) that actually resolve to the stated record.
Code Availability & Versioning	12%	Whether the simulation code is publicly archived, versioned, and reachable at a stable location, with an explicit license.
Code-to-Data Traceability	15%	Whether each computational step can be traced back to the specific data source and citation it depends on.
Simulation Derivation Clarity	14%	Whether model parameters, assumptions, and update rules are derived transparently rather than asserted, so the simulation can be re-derived.
Reproducibility Package Quality	7%	Whether a runnable package (environment spec, seeds, instructions) lets an independent reader reproduce the reported results.
Information-Theoretic Rigor	25%	Whether the paper formalizes and quantifies the information content of the system it models. Orthogonal to transparency; non-applicable topics are scored at a neutral baseline (~50).

2.3 Conformance to good data and reproducibility practice

Six of the seven dimensions measure whether a simulation accords with established good practice for data and software. Data Disclosure and Dataset Resolvability test the FAIR ideal that data be findable and accessible through identifiers that genuinely resolve — a declared dataset with no resolvable accession scores poorly even if the prose claims openness. Code Availability & Versioning and Reproducibility Package Quality test whether the software is archived, licensed, versioned, and packaged so an independent reader can run it. Code-to-Data Traceability and Simulation Derivation Clarity test the chain of custody from result back to source: can each computational step and model parameter be traced to the specific data and citation it depends on, rather than asserted? Crucially, these scores are anchored to verified external signals — a resolved DOI, a real license file detected in the repository — and not to the paper's self-description; the framework does not credit unverifiable claims, and does not invent gaps it cannot substantiate.

2.4 Code-to-data traceability

Beyond scoring, BioEval can accept the simulation's source code directly: the model segments the code and maps each segment to the data source and citation it relies on, with a high/medium/low confidence rating. This makes the dependency between a result and its inputs explicit and auditable, which is the operational meaning of reproducibility for a simulation.

3 Information-Theoretic Rigor

The seventh dimension is orthogonal to the other six: it measures scientific-content rigor rather than transparency. Many papers in this domain model communication and coordination systems — pheromone stigmergy in ants and termites, waggle-dance signaling in bees, phase coupling in firefly and Kuramoto

synchronization — where information flow is not a side effect but the phenomenon itself. For such systems, rigor means formalizing that information: the Shannon entropy of agent state or signal distributions, the mutual or transfer entropy that captures directed information flow between agents, and the channel capacity or communication bit rate of the signaling mechanism (bits per pheromone deposit, bits per dance, bits per second), including how these scale with colony size and signal-to-noise.

The guideposts reward papers that actually do this mathematics and penalize papers where communication is central yet treated purely mechanistically. To avoid punishing work for which information theory is simply not applicable — a pure phylogenetics or sequence-alignment pipeline, say — such topics are scored at a neutral midpoint (~50) rather than zero, so non-applicability neither rewards nor punishes. Because the dimension is orthogonal to transparency, a paper can be exemplary in data practice yet score low here, and that contrast is itself the finding this report foregrounds.

4 Evaluation Corpus and Method

We evaluated 14 computational-simulation projects spanning ant, bee, termite, and firefly systems and ant-colony optimization, drawn from the insect-swarm simulation series. Every project was scored under a single rubric version (v0.9.0) so the results are mutually comparable. Scores are assigned by the language model against the fixed guideposts of Section 2, grounded in the external evidence gathered during analysis. In the table below the overall column is recomputed as the exact weighted mean of the seven dimensions, so each row is reproducible from the published weights.

5 Results

5.1 Score summary

Simulation project	Ovr	DD	DR	CA	TR	SC	RP	IT
BeeStack: An Evidence-Typed Scaffold for Whole-Colo…	**55.6**	72	68	63	70	58	42	30
Bee Swarm Simulation	**53.0**	72	68	58	62	42	55	32
The Ant Stack: Reproducible scientific workspace fo…	**44.7**	52	48	62	55	42	58	22
Ant Simulation — Pygame Stigmergy	**40.4**	62	58	63	45	28	42	12
Ant Colony Optimization — React/Canvas	**39.2**	52	55	72	38	22	62	12
Rust Ant Colony Simulation	**39.0**	62	70	52	35	28	48	10
Locas Ants — Lua/Love2D Ant Colony	**37.1**	42	48	72	30	22	55	20
ACO Robot Path Planning (Python)	**28.6**	22	38	48	35	28	12	20
Firefly Synchronization Simulation (JS)	**28.0**	28	35	48	30	22	25	18
Hardware-Accelerated Ant Colony Swarm (C++)	**27.9**	28	35	52	22	18	35	20
Termite Colony — Unity 3D Multiagent	**27.5**	42	38	38	22	30	22	12
Ants Sandbox (TypeScript/web)	**23.2**	22	28	62	15	12	38	10
Symbiants — Ant Colony Sim (Rust)	**22.1**	18	30	58	12	15	35	10
swarm-abc — Artificial Bee Colony Simulation	**18.6**	18	22	32	12	18	8	18

DD Data Disclosure · DR Dataset Resolvability · CA Code Availability · TR Code-to-Data Traceability · SC Simulation Clarity · RP Reproducibility Package · IT Information-Theoretic Rigor. Overall is the weighted mean of all seven dimensions. Scores are rubric v0.9.0 (0–100).

5.2 Observations

Three patterns recur across the corpus. Transparency is uneven: code availability tends to be the strongest dimension — most projects publish a repository — while reproducibility-package quality and code-to-data traceability are consistently the weakest, meaning the code exists but cannot easily be re-run or traced to its inputs. The headline finding is the systematically low Information-Theoretic Rigor across projects whose entire subject is communication and coordination: even strong, transparent simulations rarely quantify the

information their agents exchange. The orthogonality of the rubric makes this visible — high transparency and low information-theoretic rigor coexist in the same papers, marking a concrete and addressable opportunity for the field.

5.3 Per-project synopsis

BeeStack: An Evidence-Typed Scaffold for Whole-Colony Honeybee Simulation — overall 55.6 (IT 30)

BeeStack demonstrates commendable transparency infrastructure: the software is deposited on Zenodo with a DOI and versioned GitHub release, and the repository source files contain numerous resolved DOIs linking to empirical datasets (e. g.

Principal gap: GitHub API detects NO license file in the repository itself, contradicting the MIT declaration on Zenodo; this creates a legal ambiguity for reuse directly from GitHub

Bee Swarm Simulation — overall 53.0 (IT 32)

This is a browser-based agent simulation with all source files publicly available on GitHub and no external build dependencies, making it straightforwardly runnable for anyone with a browser. The citation-backed mode documents its biological parameter sources against a credible set of published references (von Frisch 1967, Seeley 1995, Couvillon 2019, Schurch 2021, Menzel 2023), but the heuristic mode's hand-tuned parameter values are not documented, and neither mode provides pinned numerical parameter tables traceable to specific passages.

Principal gap: No software license is present in the repository, creating legal ambiguity around reuse and reproduction

The Ant Stack: Reproducible scientific workspace for ant-inspired simulation and complexity energetics — overall 44.7 (IT 22)

The Ant Stack presents a modular simulation framework with a reasonably structured GitHub repository (antstack_core), a passing test suite (676 tests), documented CLI entry points, manifest-driven artifact generation with SHA-256 checksums, and YAML-based configuration files — all positive transparency signals. However, the paper is primarily a design specification and roadmap rather than a completed computational study: key contributions (species presets, evaluation benchmarks, baseline seeds) are described as future work, no public repository URL is provided, no archived DOI for the code exists, and foundational datasets (FORMINDEX, MetaInformAnt, Blue Brain, Eyewire) are cited without accession numbers or data availability statements.

Principal gap: No public URL for the GitHub repository is provided in the paper or README, making direct discovery by readers non-trivial

Ant Simulation — Pygame Stigmergy — overall 40.4 (IT 12)

This is a publicly accessible GitHub simulation repository with an MIT license, a minimal README, and a requirements. txt dependency manifest, making it partially reproducible.

Principal gap: Dependency versions are not pinned in requirements.txt, making exact environment reconstruction uncertain across future Python/library versions

Ant Colony Optimization — React/Canvas — overall 39.2 (IT 12)

This project is a self-contained React/Canvas ant simulation with no external datasets, and its generative inputs (JSON map, sprite sheets, tileset) are all bundled within the public GitHub repository under an MIT license with installation instructions provided. However, reproducibility is weakened by a placeholder clone URL, absence of a pinned commit or versioned release, no surfaced dependency versions in the README, and hard-coded simulation parameters (ant states, movement rules) with no cited sources or documented rationale.

Principal gap: No pinned commit hash or tagged release is cited, so the exact code state cannot be unambiguously referenced

Rust Ant Colony Simulation — overall 39.0 (IT 10)

This is a public GitHub repository for an ant colony simulation written in Rust using the Bevy game engine, with no external datasets (appropriately so for a generative simulation). The code is publicly accessible with a Cargo.

Principal gap: No software license is present, creating legal ambiguity for reuse or reproduction

Locas Ants — Lua/Love2D Ant Colony — overall 37.1 (IT 20)

This is a GitHub-hosted Lua/Löve2D ant colony simulation with 161 commits, 6 versioned releases, an MIT license, and a basic README providing installation instructions — a reasonable open-source release but far below research-grade reproducibility standards. Because the project uses no external biological or experimental datasets, evaluation pivots to disclosure of generative inputs (rules, parameters, initial conditions), which are entirely embedded in source code with no documentation, configuration files, or parameter tables provided in the README or supplementary materials.

Principal gap: Simulation parameters (pheromone evaporation rate, deposit amounts, ant sensing radius, colony size, movement rules) are embedded in source code with no externalized configuration file, parameter table, or documentation

ACO Robot Path Planning (Python) — overall 28.6 (IT 20)
This repository provides a publicly accessible Python implementation (PathPlanning. py) of an ACO-based robot path planning algorithm, licensed under GPL-3.

Principal gap: No dependency manifest (requirements.txt, setup.py, or equivalent) is present, making environment reconstruction manual and error-prone

Firefly Synchronization Simulation (JS) — overall 28.0 (IT 18)
This is a GitHub-hosted firefly synchronization simulation with publicly accessible code but extremely limited reproducibility infrastructure: no license, no versioned release or pinned commit, a minimal README (~42 bytes), no CI/tests, and no archived snapshot (e. g.

Principal gap: No software license detected, making legal reuse and redistribution ambiguous

Hardware-Accelerated Ant Colony Swarm (C++) — overall 27.9 (IT 20)
This repository presents a hardware-accelerated ant colony swarm simulation using CUDA and OpenGL, with source code publicly available on GitHub (12 stars, 38 commits, last pushed May 2026). However, the repository lacks a license, version tags, releases, dependency manifests with pinned versions, CI/CD, and tests, and the README is minimal (~1264 bytes) with no documentation of swarm parameters, evolutionary algorithm configuration, grid size, agent counts tested, or initial conditions.

Principal gap: No software license is present in the repository, preventing legal reuse or redistribution

Termite Colony — Unity 3D Multiagent — overall 27.5 (IT 12)
This is a GitHub README-only submission for a Unity/C# termite colony simulation with no associated research paper, journal article, or formal methods section. The repository is public under MIT license with 38 commits but has no versioned releases, no dependency manifest, no CI, no tests, and a README that was flagged as missing/empty by the live GitHub API — contrasting with the scraped content provided.

Principal gap: The clone URL in the README (https://github.com/Dougarasu/TrabalhoIA.git) does not match the actual repository URL, breaking reproducibility for anyone following instructions literally

Ants Sandbox (TypeScript/web) — overall 23.2 (IT 10)
This repository is a browser-based ant colony simulation written in TypeScript/Vue, publicly available on GitHub under MIT license with basic build instructions, but it falls short on nearly every scientific reproducibility dimension. There are no external datasets (appropriate for a simulation), but the generative parameters, agent rules, pheromone decay constants, and initial conditions are entirely undocumented — the README is a minimal ~490-byte stub with no parameter descriptions or citations.

Principal gap: Simulation parameters (pheromone decay rates, evaporation constants, ant sensing radii, colony size, food quantities, etc.) are not documented anywhere in the README or linked configuration files

Symbiants — Ant Colony Sim (Rust) — overall 22.1 (IT 10)
This submission is a GitHub repository for a Rust/Bevy ant colony simulation game, not a scientific paper in the conventional sense — it lacks any scientific manuscript, methodology section, or quantitative analysis. The repository is publicly accessible under dual Apache-2.

Principal gap: No published versioned releases (0 releases confirmed via GitHub API) and no git tags, making it impossible to cite a specific reproducible snapshot

swarm-abc — Artificial Bee Colony Simulation — overall 18.6 (IT 18)
This repository contains a bare-minimum public GitHub repository for an Artificial Bee Colony simulation implemented in HTML/JavaScript/CSS, but it lacks virtually all elements required for scientific reproducibility. There is no README, no license, no dependency manifest, no versioned release or tagged commit, no documented parameters, no CI, and no archived snapshot (e.

Principal gap: No README file is present, providing zero guidance on how to run, configure, or interpret the simulation

The complete evaluation for every project — full summary, findings, gaps, and recommendations — is reproduced in Appendix A.

6 Discussion

BioEval treats reproducibility as a property that must be demonstrated through resolvable evidence rather than claimed in prose, and it adds a second axis — information-theoretic rigor — that is specific to the communication systems this literature studies. The two axes are complementary: good data practice makes a result checkable, while information-theoretic formalization makes it meaningful for systems whose subject is information. Using a language model as the evaluator is what makes assessment at this breadth tractable; fixed guideposts, evidence grounding, and a versioned rubric are what keep it disciplined.

7 Limitations

Scores are produced by a language model and tend to snap to the discrete tiers defined by the guideposts; feeding real, per-item external signals (resolved accessions, live repository facts) is what differentiates otherwise-similar papers. The rubric is pre-1.0 and still under reviewer validation, so weights and guideposts may change between versions — hence the stamped rubric version on every evaluation. The corpus is domain-specific (insect-swarm and agent-based simulation), and the information-theoretic dimension is calibrated for systems where information flow is central; its neutral-baseline treatment of non-applicable topics is a deliberate, conservative choice rather than a measurement.

8 Availability and Reproducibility

BioEval is released under the MIT license. The source repository is hosted at github.com/fractastical/bioinformatics-eval, and this Zenodo deposition bundles the present report together with a complete archive of the source code at the corresponding revision. The system is built as a pnpm monorepo (React + Vite frontend, Express 5 API, PostgreSQL with Drizzle ORM, contract-first OpenAPI/Orval codegen) and uses Anthropic Claude through Replit's AI integrations. The evaluation rubric and the software are co-versioned at v0.9.0.

References

[1] Shannon, C. E. (1948). A Mathematical Theory of Communication. Bell System Technical Journal, 27, 379–423, 623–656.

[2] Grassé, P.-P. (1959). La reconstruction du nid et les coordinations interindividuelles: la théorie de la stigmergie. Insectes Sociaux, 6, 41–80.

[3] Dorigo, M., & Stützle, T. (2004). Ant Colony Optimization. MIT Press.

[4] Kuramoto, Y. (1984). Chemical Oscillations, Waves, and Turbulence. Springer.

[5] Schreiber, T. (2000). Measuring Information Transfer. Physical Review Letters, 85(2), 461–464.

[6] Wilkinson, M. D., et al. (2016). The FAIR Guiding Principles for scientific data management and stewardship. Scientific Data, 3, 160018.

[7] Sandve, G. K., et al. (2013). Ten Simple Rules for Reproducible Computational Research. PLoS Computational Biology, 9(10), e1003285.

A Full Per-Project Evaluations

This appendix reproduces the complete evaluation BioEval produced for each project, in the same descending-overall order as Section 5. For every project it lists the per-dimension scores and the full narrative — summary, findings, gaps, and recommendations — exactly as generated under rubric v0.9.0, so the synopses in Section 5.3 can be read against their complete supporting detail.

A.1 BeeStack: An Evidence-Typed Scaffold for Whole-Colony Honeybee Simulation

Overall 55.6 · DD 72 DR 68 CA 63 TR 70 SC 58 RP 42 IT 30 · rubric v0.9.0

Summary

BeeStack demonstrates commendable transparency infrastructure: the software is deposited on Zenodo with a DOI and versioned GitHub release, and the repository source files contain numerous resolved DOIs linking to empirical datasets (e.g., in docs/beebrain_data_pipeline.md and manuscript/references.bib), supporting data-to-code traceability. However, the GitHub repository critically lacks a detected license file (the Zenodo record claims MIT but GitHub API reports none), has no CI/CD, a minimal README (~259 bytes), and no pinned dependency versions, substantially limiting practical reproducibility. The paper's central subject—waggle-dance communication and swarm coordination—calls strongly for information-theoretic quantification (bits per dance, channel capacity, mutual information between foragers), but the framing appears mechanistic and scaffold-oriented rather than formally quantifying these measures. One DOI (10.1186/s12864-019-5639-3) is unresolved, and several neural subsystem datasets and BEEHAVE integration resources lack explicit accession numbers in the record itself.

Findings

- Source code is publicly available on GitHub at https://github.com/docxology/BeeStack with a versioned v1.0.0 tagged release
- Software is archived on Zenodo with persistent DOI 10.5281/zenodo.20420557 and downloadable source tarball with MD5 checksum
- Repository source files (docs/beebrain_data_pipeline.md, manuscript/03_materials_and_source_provenance.md, manuscript/references.bib) contain numerous resolvable DOIs linking to empirical datasets, including figshare (10.6084/m9.figshare.24715977.v1), multiple Dryad records (10.5061/dryad.*), and journal articles
- 9 of 10 scanned DOIs from repository source files resolve publicly; coverage is broad across neuroanatomy, genomics, and behavioral datasets
- Known biological data gaps are explicitly catalogued as a design principle rather than hidden, and fidelity is declared as a per-module property
- A pyproject.toml dependency manifest is present, enabling environment reconstruction
- A tests directory is present in the repository
- The combined manuscript PDF is publicly available for download alongside the source archive
- MIT License is declared on Zenodo, enabling open reuse
- The scaffold explicitly traces parameters from configuration to artifact to manuscript, with config.yaml containing the self-referential Zenodo DOI

Gaps

- GitHub API detects NO license file in the repository itself, contradicting the MIT declaration on Zenodo; this creates a legal ambiguity for reuse directly from GitHub
- README is minimal (~259 bytes), providing insufficient documentation for independent setup and execution
- No CI/CD (GitHub Actions or equivalent) is detected, meaning automated reproducibility testing is absent
- No Dockerfile or containerization is present, requiring manual environment reconstruction
- One DOI (10.1186/s12864-019-5639-3, listed in beebrain_data_pipeline.md) is unresolved/not found in Crossref or DataCite
- Neural subsystem datasets (antennal lobe, mushroom body, central complex) and waggle-dance behavioral datasets are not enumerated with accession numbers in the Zenodo record itself, only within repository markdown files
- pyproject.toml dependency versions are not confirmed to be pinned, risking environment drift
- No example input data or small test fixtures with checksums are confirmed present for end-to-end workflow verification
- BEEHAVE integration is referenced but no dataset or code link for BEEHAVE is provided in the record or confirmed in the repository scan
- FlyBody and MuJoCo component version numbers and configuration files are not specified in the Zenodo record
- Information-theoretic quantities central to waggle-dance and swarm communication (bits per dance, channel capacity, mutual information between foragers) are not formally quantified
- [Critic] No CI/CD workflows are present in the repository, meaning there is no automated evidence that the software installs or runs correctly across environments — the validation rate described as a 'config-band self-test measure' has no continuously verified execution record.

- [Critic] One DOI cited in the repository source files (10.1186/s12864-019-5639-3) did not resolve in Crossref or DataCite, leaving one referenced resource unverified.

Recommendations

- Add a LICENSE file directly to the GitHub repository root to resolve the discrepancy with the Zenodo MIT declaration
- Expand the README to include installation steps, a minimal working example, and a description of each of the five simulation layers
- Add GitHub Actions CI that runs the test suite on push, providing automated reproducibility assurance
- Provide a Dockerfile or conda environment file with pinned dependency versions to enable one-command environment setup
- Resolve or replace DOI 10.1186/s12864-019-5639-3, which is currently unresolvable; provide an alternative access path or note its unavailability
- Create a dedicated data manifest (e.g., data/DATASETS.md) listing every empirical dataset with accession number, version, access method, and role in the simulation
- Include small bundled test fixtures with checksums so the test suite can verify end-to-end pipeline integrity without downloading external data
- For the waggle-dance and swarm coordination modules, formally quantify information-theoretic measures: Shannon entropy of dance angle/distance distributions, mutual information between dancer and recruiter, and estimated channel capacity in bits per waggle run
- Pin all dependency versions in pyproject.toml and regenerate a lock file (e.g., uv.lock or poetry.lock) for full environment reproducibility
- Add version numbers and configuration file references for FlyBody and MuJoCo to the Zenodo record and manuscript materials-and-methods section

A.2 Bee Swarm Simulation

Overall 53.0 · DD 72 DR 68 CA 58 TR 62 SC 42 RP 55 IT 32 · rubric v0.9.0

Summary

This is a browser-based agent simulation with all source files publicly available on GitHub and no external build dependencies, making it straightforwardly runnable for anyone with a browser. The citation-backed mode documents its biological parameter sources against a credible set of published references (von Frisch 1967, Seeley 1995, Couvillon 2019, Schurch 2021, Menzel 2023), but the heuristic mode's hand-tuned parameter values are not documented, and neither mode provides pinned numerical parameter tables traceable to specific passages. The repository lacks a license, versioned releases or tags, dependency manifests, tests, CI, and a persistent archive (e.g., Zenodo DOI), which substantially limits exact reproducibility of any specific simulation run. While the simulation models the waggle-dance information channel — a paradigmatic information-theoretic communication system — Shannon entropy is invoked only as a labeling device for 'Env Data Volume' and 'per-run payload'; no channel capacity, mutual information, or bit-rate derivation is presented or measured.

Findings

- Source code for all simulation components (app.js, bee_swarm_live_sim.html, styles.css) is publicly accessible on GitHub with no build step or external runtime dependencies required
- README documents a BeeStack trace JSON schema with field-level specification, enabling external trace replay
- Citation-backed mode explicitly aligns directional encoding (von Frisch 1967), foraging behavior (Seeley 1995), non-linear distance calibration (Schurch et al. 2021), follower sampling (Ai et al. 2019), and recruit inference (Menzel et al. 2023) with named literature sources
- Shannon (1948) and Fisher (1935/circular statistics) are cited as framing references for on-screen metric displays
- Thermal defensive balling temperature (~47°C) is precisely sourced to Ono et al. (1995) in Nature, demonstrating parameter-level citation
- The Export Results JSON feature enables capture of run summaries and event logs, aiding post-hoc reproducibility
- Repository is actively maintained (last push 2026-06-05) with an open issue indicating community engagement
- Simulation offers two clearly distinguished information model modes (citation-backed vs. heuristic), making modeling assumptions explicit at runtime

Gaps

- No software license is present in the repository, creating legal ambiguity around reuse and reproduction
- No versioned releases or Git tags exist, so there is no mechanism to pin a specific reproducible simulation version
- Heuristic mode's hand-tuned parameter values are not documented anywhere in the README or code comments visible in the repository listing
- No dependency manifest (package.json, requirements.txt, etc.) exists, though runtime dependencies are absent; build-tooling or future dependencies would be untracked
- No automated tests or CI pipeline detected, meaning correctness of the simulation logic cannot be independently verified

- No persistent archived snapshot (e.g., Zenodo DOI) exists, so long-term citability of a specific version is not guaranteed
- Specific numerical parameter values (e.g., waggle run duration–distance calibration constants, follower sampling probabilities) are not mapped to exact table or equation numbers in cited references
- Random seed initialization is not described or configurable in the documented interface, making stochastic run-to-run reproducibility uncertain
- The 'communication manifold' and 'mini heat panels' metrics are mentioned but their computational definitions are not documented
- [Critic] The repository has no license detected. Depending on the paper's claims about open availability or reuse, the absence of a license means the code is technically under default copyright and cannot be freely reused, forked, or redistributed by others—potentially undermining the practical openness implied by 'publicly available on GitHub.'
- [Critic] The repository has only 1 star and 1 open issue, with a last push dated 2026-06-05, suggesting very recent creation close to (or after) submission. If the paper implies the simulation has been publicly available and community-validated for some period, the recency and minimal community engagement do not support that implication.
- [Critic] No archived snapshot (e.g., Zenodo DOI) is detectable or cited, meaning long-term provenance and citability of the exact code version used in the paper are not assured by any persistent identifier.

Recommendations

- Add an open-source license (e.g., MIT or Apache 2.0) to the repository to permit legal reuse and citation
- Create a versioned GitHub release or annotated Git tag for any simulation used in a publication, and archive it on Zenodo to obtain a persistent DOI
- Document all heuristic-mode parameter values in a parameter table in the README, with rationale or provenance for each hand-tuned constant
- Add a pinned random seed option exposed through the UI and documented in the README, so stochastic runs can be exactly reproduced
- Provide a supplementary parameter table mapping each citation-backed constant to the specific figure, table, or equation in the cited reference
- Define the computational formulas for 'Env Data Volume', 'Reality Equivalent', 'communication manifold', and directional/distance coherence metrics explicitly in the README or a supplementary methods document
- Formalize the information-theoretic analysis: compute the channel capacity of the waggle dance signaling channel in bits per run (using direction variance and distance calibration uncertainty from cited sources), and measure Shannon entropy of the agent state distribution over simulation time
- Add a minimal test suite (e.g., a headless Node.js or Playwright script) that runs one simulation step and checks key output invariants, enabling CI-based regression testing

A.3 The Ant Stack: Reproducible scientific workspace for ant-inspired simulation and complexity energetics

Overall 44.7 · DD 52 DR 48 CA 62 TR 55 SC 42 RP 58 IT 22 · rubric v0.9.0

Summary

The Ant Stack presents a modular simulation framework with a reasonably structured GitHub repository (antstack_core), a passing test suite (676 tests), documented CLI entry points, manifest-driven artifact generation with SHA-256 checksums, and YAML-based configuration files — all positive transparency signals. However, the paper is primarily a design specification and roadmap rather than a completed computational study: key contributions (species presets, evaluation benchmarks, baseline seeds) are described as future work, no public repository URL is provided, no archived DOI for the code exists, and foundational datasets (FORMINDEX, MetaInformAnt, Blue Brain, Eyewire) are cited without accession numbers or data availability statements. Most critically, pheromone field parameters lack concrete default values, simulation seeds are not reported, and the paper treats information flow — the central phenomenon of stigmergic ant communication — almost entirely mechanistically with no Shannon entropy, mutual information, or channel capacity calculations.

Findings

- GitHub repository (ant_stack) exists with a structured Python package (antstack_core), CLI entry points (antstack-build, antstack-ce, run-all-antstack), and a verified test suite (676 passed)
- Canonical run artifacts are generated with a documented single command and include manifest.json with SHA-256 checksums and provenance files under outputs/<run_id>/
- YAML/JSON configuration files (configs/run_all_antstack.example.yaml, papers/complexity_energetics/manifest.example.yaml) provide manifest-driven, version-pinned workflow definitions
- Physics simulation engines (MuJoCo, PyBullet) are explicitly cited with references to original papers, providing traceability for the physical layer
- Virtual Fly Brain (VFB) is explicitly cited as the template resource for neural circuit nomenclature and parameters, grounding the AntBrain layer

- Neural simulation engine candidates (Brian2, Nengo, SpikingJelly) are named in implementation notes
- A Reproducibility Checklist appendix is included in the paper
- Dependency management uses both uv (Python) and bun (Node), with sync commands documented, indicating a structured environment specification
- The repository enforces folder documentation coverage via tools/ensure_folder_docs.py --check, indicating systematic documentation discipline
- Key simulation parameters (physics step, control frequency, neuron count, sensor noise) are explicitly stated in the paper body

Gaps

- No public URL for the GitHub repository is provided in the paper or README, making direct discovery by readers non-trivial
- No archived code DOI (e.g., Zenodo snapshot) is present, meaning the repository has no persistent, citable, versioned archive
- No pinned commit hash or tagged release is mentioned; reproducibility depends on the current HEAD of an unarchived repo
- Foundational datasets (FORMINDEX, MetaInformAnt, Blue Brain Project, Eyewire) are listed as inspirations without accession numbers, DOIs, or data availability statements
- Evaluation benchmarks and baseline species presets are described as future contributions ('will be implemented'), not yet available
- Pheromone field parameters (decay ;²À diffusion D) are defined symbolically but no concrete default numerical values are provided in the paper
- Simulation random seeds are not reported anywhere in the paper for any experiment or figure
- No explicit software license is mentioned in the paper text or README excerpt
- The referenced Resources.md file for core literature is not included in or linked from the paper
- Pheromone-mediated communication — the central coordination mechanism — is never quantified information-theoretically (no channel capacity, mutual information, or entropy calculations)
- Full bibliographic details for several key parameter-grounding citations (Wilson 2013, Caron et al. 2013, Seelig & Jayaraman 2015, Wehner 2003) are absent from the paper body and deferred to an external file
- [Critic] No empirical validation or benchmark result is cited to confirm that the stated simulation parameters (9G@ = 1 ms, 100 Hz, <0 " [0.01, 0.05]) produce biologically plausible or reproducible ant locomotion behavior, leaving the 'recommended defaults' claim unsupported by outcome evidence.
- [Critic] The SHA-256 checksum manifest is described structurally but no example checksum or sample manifest.json is provided, making it impossible to verify that the provenance pipeline was actually executed rather than only designed.

Recommendations

- Provide the explicit public GitHub URL in the paper abstract or data availability statement, and create a Zenodo-archived release with a citable DOI
- Tag a versioned release (e.g., v0.1.0) and pin the commit hash in the paper to ensure exact reproducibility
- Add a formal Data Availability Statement listing each external dataset (FORMINDEX, MetaInformAnt, etc.) with accession numbers, URLs, and access dates
- Publish concrete numerical default values for all pheromone field parameters (;²À D, and grid resolution) in a parameter table within the paper
- Report random seeds for all simulations, figures, and evaluation runs in a reproducibility appendix or config file referenced from the paper
- Include an explicit open-source license (e.g., MIT, Apache 2.0) in the repository and state it in the paper
- Add at least one information-theoretic analysis: compute the Shannon entropy of the pheromone field state distribution, or the mutual information between pheromone signal and subsequent agent action, to rigorously characterize the communication channel
- Make the species presets, evaluation benchmarks, and baseline seeds available now (even as preliminary defaults) rather than deferring entirely to future work
- Include a complete, self-contained bibliography in the paper rather than deferring to external Resources.md files
- Add a CI badge and pinned dependency lock files (uv.lock, bun.lockb) to the repository to strengthen environment reproducibility guarantees

A.4 Ant Simulation — Pygame Stigmergy

Overall 40.4 · DD 62 DR 58 CA 63 TR 45 SC 28 RP 42 IT 12 · rubric v0.9.0

Summary

This is a publicly accessible GitHub simulation repository with an MIT license, a minimal README, and a requirements.txt dependency manifest, making it partially reproducible. However, it critically lacks pinned dependency versions, no tagged releases or archived DOI (e.g., Zenodo), no CI/testing infrastructure, and no empirical or literature-based justification for any of the simulation parameters. The 'Mental Fatigue' mechanism is described as a novel discovery without reference to prior ACO or swarm intelligence literature, and no information-theoretic framing is applied despite pheromone stigmergy being a canonical information-theoretic signaling system. The simulation generates its own data, which partially compensates for the absence of external datasets, but generative parameters lack documented default values in the README and have no calibration basis.

Findings

- Repository is publicly accessible on GitHub under the MIT License, confirmed via live API signals
- A requirements.txt dependency manifest is present, listing Pygame, NumPy, and SciPy
- The simulation uses no external datasets; all data is generated at runtime, making accession numbers legitimately inapplicable
- Key simulation parameters (ANT_COUNT, EVAPORATION_RATE, NEST_GRAVITY_STRENGTH, MAX_PATIENCE, PANIC_DURATION) are exposed as top-level constants in sim.py, enabling manual experimentation
- A demo video (demo.mp4) is included in the repository to illustrate emergent behavior
- A README is present (~3660 bytes) with clear installation and run instructions
- The generative rules ('Laws of Physics') are described conceptually in the README, providing partial traceability of simulation logic

Gaps

- Dependency versions are not pinned in requirements.txt, making exact environment reconstruction uncertain across future Python/library versions
- No tagged releases, no published GitHub releases, and no DOI/Zenodo archive exist, limiting persistent citability and snapshot reproducibility
- No CI (GitHub Actions) or automated tests are present to verify simulation correctness after changes
- Default numeric values for all simulation parameters are absent from the README; users must open sim.py to discover them
- No empirical basis, calibration procedure, or prior literature citations are provided for any parameter values (e.g., why specific evaporation rate or patience threshold)
- The 'Mental Fatigue' and 'Panic' mechanism is claimed as a novel discovery but cites no prior swarm intelligence or ACO literature for context or differentiation
- No operating system or hardware requirements are specified, and no random seed is documented or exposed, making exact run replication impossible
- No information-theoretic analysis is performed: pheromone stigmergy is a well-studied communication channel yet Shannon entropy, mutual information, or channel capacity are never discussed or quantified
- [Critic] No version pinning is confirmed for dependencies: the requirements.txt exists but no evidence shows whether specific versions are pinned, which affects reproducibility of simulation behavior across library versions.
- [Critic] No tests directory or CI workflow is present, meaning there is no automated verification that sim.py executes correctly with the listed dependencies.
- [Critic] The repository has 0 published releases and 0 tags, so there is no versioned snapshot of the code corresponding to any specific paper submission or analysis.
- [Critic] The single open issue is unresolved; its content is unknown and could indicate a known defect in the simulation code.

Recommendations

- Pin all dependency versions in requirements.txt (e.g., pygame==2.5.2, numpy==1.26.4, scipy==1.12.0) to ensure reproducible environment setup
- Create a tagged release (e.g., v1.0.0) and deposit the repository to Zenodo to obtain a persistent DOI for citable, archived reference
- Add a table of default parameter values with brief rationale to the README, so readers can understand the baseline configuration without reading source code
- Add a --seed CLI argument and document the default random seed so that individual simulation runs can be exactly reproduced
- Cite relevant prior literature on Ant Colony Optimization (Dorigo & Gambardella 1997), stigmergy, and patience/exploration mechanisms to contextualize the 'Mental Fatigue' contribution
- Add at minimum one GitHub Actions CI workflow that runs a short headless simulation (e.g., 100 steps) and checks for no runtime errors, providing basic correctness assurance

- Quantify at least one information-theoretic property of the pheromone communication system, such as the Shannon entropy of the pheromone field over time or the mutual information between ant positions and pheromone gradients, to elevate scientific rigor
- Specify minimum Python version and tested operating systems in the README to reduce environment ambiguity

A.5 Ant Colony Optimization — React/Canvas

Overall 39.2 · DD 52 DR 55 CA 72 TR 38 SC 22 RP 62 IT 12 · rubric v0.9.0

Summary

This project is a self-contained React/Canvas ant simulation with no external datasets, and its generative inputs (JSON map, sprite sheets, tileset) are all bundled within the public GitHub repository under an MIT license with installation instructions provided. However, reproducibility is weakened by a placeholder clone URL, absence of a pinned commit or versioned release, no surfaced dependency versions in the README, and hard-coded simulation parameters (ant states, movement rules) with no cited sources or documented rationale. The simulation treats pheromone stigmergy and agent coordination—phenomena with a natural information-theoretic framing—in a purely mechanistic, early-stage manner with no quantification of information flow, channel capacity, or entropy of agent states. No pathfinding is implemented and the project is explicitly described as a learning exercise, so scientific rigor is minimal across all dimensions.

Findings

- Source code is publicly available on GitHub (cfrBernard/ant-colony-optimization) under the MIT License
- All generative assets (JSON map from Tiled Map Editor, ant sprite sheet, tileset image) are included directly in the repository
- Installation and execution instructions are provided (npm install / npm start / localhost:3000)
- package.json and package-lock.json are present, enabling dependency resolution
- Current limitations (no pathfinding, no advanced AI) are explicitly and honestly stated
- Repository language breakdown is transparent (JavaScript 82.6%, HTML 11.3%, CSS 6.1%)
- The project is released under the MIT License allowing free use and modification

Gaps

- No pinned commit hash or tagged release is cited, so the exact code state cannot be unambiguously referenced
- The README clone URL uses a placeholder '<your-username>' instead of the actual repository owner, creating a discrepancy
- Exact dependency versions are not surfaced in the README; users must inspect package.json/package-lock.json directly
- Simulation parameters (ant states IDLE, WALK, movement rules) are hard-coded in Ant.js with no cited source or documented rationale
- No tests or CI configuration are present to verify correctness of the simulation logic
- No Zenodo or equivalent archival DOI exists, so long-term citability is not guaranteed
- Pheromone/stigmergy logic and initial conditions are not formally specified anywhere in the documentation
- No information-theoretic analysis of agent communication, pheromone channel capacity, or entropy of state distributions is present despite ant colony coordination being a canonical domain for such analysis

Recommendations

- Tag a versioned release (e.g., v0.1.0) or pin a specific commit hash in the README to ensure exact reproducibility
- Fix the placeholder clone URL in the README to use the actual repository path (cfrBernard/ant-colony-optimization)
- Archive the repository to Zenodo or figshare to obtain a citable DOI for long-term persistence
- Document all simulation parameters (state transition probabilities, movement speed, pheromone decay rate if/when implemented) in a dedicated parameters section with units
- Surface key dependency versions (React version, Node.js version) explicitly in the README rather than requiring inspection of package.json
- Add a brief parameter-rationale section explaining why IDLE/WALK states are defined as they are, even if empirically chosen
- Add at minimum a smoke test (e.g., a Jest test confirming the simulation initializes without errors) to support automated verification
- If the project is extended to include pheromone dynamics, consider quantifying pheromone signal entropy or mutual information between ant states to provide scientific grounding

A.6 Rust Ant Colony Simulation

Overall 39.0 · DD 62 DR 70 CA 52 TR 35 SC 28 RP 48 IT 10 · rubric v0.9.0

Summary

This is a public GitHub repository for an ant colony simulation written in Rust using the Bevy game engine, with no external datasets (appropriately so for a generative simulation). The code is publicly accessible with a Cargo.toml dependency manifest and basic build instructions, but lacks a versioned release, license, CI pipeline, tests, pinned commit reference, and any archived snapshot. Simulation parameters are hard-coded in src/configs.rs with no citations to literature or biological sources, and the README is minimal (~977 bytes). As a simulation where pheromone-based stigmergy and information flow are the core phenomena, the work contains no information-theoretic formalization, quantification, or even conceptual framing of the communication channel it models.

Findings

- Source code is publicly available on GitHub and confirmed accessible via live API signals
- Cargo.toml dependency manifest is present, enabling dependency resolution via Rust's package manager
- Basic build and run instructions are provided (cargo run --release) and are sufficient for compilation
- A configuration file (src/configs.rs) consolidates simulation parameters in one location
- No external biological or experimental datasets are used, which is appropriate for a generative agent-based simulation
- A YouTube demo/timelapse is referenced, providing qualitative behavioral validation
- The repository has community engagement (208 stars, 10 forks), suggesting the code runs as described
- The simulation's use of kdtree and query caching for performance is documented in the README

Gaps

- No software license is present, creating legal ambiguity for reuse or reproduction
- No versioned release, tag, or pinned commit hash is published, making it impossible to reproduce a specific version
- No CI/CD pipeline (GitHub Actions) is present to verify builds or run tests automatically
- No tests directory detected; there is no automated validation of simulation behavior
- Simulation parameters in src/configs.rs are not cited to any biological literature or ACO algorithm references
- The specific parameter ANT_INITIAL_PH_STRENGTH = 40.0 is suggested by trial-and-error guidance rather than derivation or citation
- No archived snapshot (e.g., Zenodo DOI) exists for long-term reproducibility
- The README is minimal (~977 bytes) with no description of the underlying algorithm, parameter sensitivity, or expected outputs
- No random seed control or determinism guarantee is documented, making exact replication uncertain
- The repository has not been updated in ~32 months with 5 open issues unaddressed
- [Critic] No license is detected in the repository, meaning reuse, modification, and redistribution rights are legally undefined — relevant if the paper implies the simulation is freely reusable by the community.
- [Critic] No published releases or tags exist, so there is no pinned, citable version of the code; the repository state at the time of any associated publication cannot be unambiguously recovered.
- [Critic] No CI/CD workflows are present, meaning the build instructions (cargo run --release) have never been automatically validated; the claim that the code compiles and runs as described rests solely on the README.
- [Critic] The repository has not been updated since 2023-10-20 (32 months ago) and has 5 open issues with no releases, raising a maintainability concern if downstream users attempt to reproduce results with updated Rust or Bevy versions.

Recommendations

- Add an open-source license (e.g., MIT or Apache 2.0) to clarify reuse permissions
- Create a tagged GitHub release or at minimum document a specific commit hash in the README for version pinning
- Archive the repository on Zenodo to obtain a citable DOI and ensure long-term accessibility
- Add citations to the ACO algorithm literature (e.g., Dorigo et al.) and biological ant foraging studies for each parameter in src/configs.rs
- Document the parameter space (ranges, sensitivity) and provide a parameter table with biological justification
- Add a GitHub Actions workflow for automated build verification on push
- Implement or document random seed control to enable deterministic reproduction of specific simulation runs
- Expand the README to include algorithm description, expected emergent behaviors, sample output images, and parameter interpretation
- Consider adding information-theoretic analysis: quantify pheromone channel capacity (bits per deposit), entropy of trail distributions, or mutual information between ant states to scientifically characterize the communication system
- Pin dependency versions in Cargo.toml or provide a Cargo.lock snapshot to ensure build reproducibility across time

A.7 Locas Ants — Lua/Love2D Ant Colony

Overall 37.1 · DD 42 DR 48 CA 72 TR 30 SC 22 RP 55 IT 20 · rubric v0.9.0

Summary

This is a GitHub-hosted Lua/Löve2D ant colony simulation with 161 commits, 6 versioned releases, an MIT license, and a basic README providing installation instructions — a reasonable open-source release but far below research-grade reproducibility standards. Because the project uses no external biological or experimental datasets, evaluation pivots to disclosure of generative inputs (rules, parameters, initial conditions), which are entirely embedded in source code with no documentation, configuration files, or parameter tables provided in the README or supplementary materials. The simulation is presented as a sandbox/game rather than a scientific study, so information-theoretic formalization of pheromone signaling — central to ant colony optimization — is entirely absent. The Python scripts (7.7% of codebase) and Makefile are undocumented, leaving portions of the build and analysis pipeline opaque.

Findings

- Source code is publicly accessible on GitHub (piXelicidio/locas-ants) with 161 commits on the master branch, confirming active development history
- MIT license is confirmed present, enabling legal reuse and redistribution
- Six tagged releases are published, including a prebuilt .love file for one-click execution without compiling from source
- README provides step-by-step installation and execution instructions for both end-users (Love2D + .love file) and developers (ZeroBrane Studio)
- Screenshots of simulation output are included in the repository, giving visual confirmation of expected behavior
- A makeLove.bat build script is present, partially automating packaging on Windows
- The project is described as a remake from scratch (not a translation), providing provenance transparency regarding the relationship to prior work
- Repository was last pushed in 2025, indicating the project is not abandoned

Gaps

- Simulation parameters (pheromone evaporation rate, deposit amounts, ant sensing radius, colony size, movement rules) are embedded in source code with no externalized configuration file, parameter table, or documentation
- No dependency manifest (e.g., rockspec, requirements.txt) with pinned version numbers for Love2D or any library in the libs/ directory
- The specific version of Love2D required is not stated anywhere in the README or repository, creating a reproducibility risk as the API has changed across versions
- The role and behavior of Python scripts (7.7% of codebase) are entirely undocumented in the README
- No CI/CD pipeline (GitHub Actions or equivalent) is present to verify the build across environments
- No tests directory detected; there is no automated validation that simulation outputs match expected behavior
- No Zenodo/DOI archive exists for long-term preservation of a specific version
- ZeroBrane Studio version is unspecified in the README
- No random seed documentation or mechanism to reproduce a specific simulation run deterministically
- No information-theoretic analysis of pheromone signaling, colony communication capacity, or emergent coordination — the central scientific phenomenon is treated purely mechanistically

Recommendations

- Externalize all simulation parameters into a Lua configuration file (e.g., config.lua) and document each parameter's meaning, units, and default value in the README
- Pin the required Love2D version explicitly in the README and ideally in a conf.lua version guard or a .love-version file
- Add a README section documenting the purpose of each Python script and Makefile target
- Create a dependency manifest listing all libraries in libs/ with their versions
- Archive a specific release on Zenodo to obtain a persistent DOI for citation in any associated publication
- Add a --seed CLI argument or config option so users can reproduce a specific simulation run exactly
- Set up a minimal GitHub Actions workflow that at least verifies the project loads without error under a pinned Love2D version
- If this simulation is intended to support a scientific publication, add a methods section quantifying key emergent properties: pheromone field entropy, convergence time as a function of colony size, or mutual information between ant positions and food source location

A.8 ACO Robot Path Planning (Python)

Overall 28.6 · DD 22 DR 38 CA 48 TR 35 SC 28 RP 12 IT 20 · rubric v0.9.0

Summary

This repository provides a publicly accessible Python implementation (PathPlanning.py) of an ACO-based robot path planning algorithm, licensed under GPL-3.0 and referencing the specific paper it simulates (Liu et al., 2017). However, the repository is extremely sparse: the README is only ~271 bytes with no usage instructions, there are no dependency manifests, no tests or CI, no versioned releases or tags, no environment specification, and no documented parameter tables or configuration files beyond the implicit reference to the source paper. As a pure simulation with no external datasets, the generative inputs (ACO parameters, grid/obstacle definitions, random seeds) are undocumented in the repository itself, severely limiting reproducibility. The information-theoretic dimension is relevant given ACO's pheromone-based communication mechanism, but the work is purely mechanistic with no quantification of information flow.

Findings

- Code is publicly accessible on GitHub under the GPL-3.0 open-source license
- A single Python source file (PathPlanning.py) implements the ACO robot path planning algorithm
- The README explicitly cites the source paper (Liu et al., 2017, Soft Computing) that the simulation replicates
- Repository has attracted 61 stars and 12 forks, indicating community interest and some degree of reuse
- No external datasets are needed for this simulation, which appropriately relies on generated grid environments

Gaps

- No dependency manifest (requirements.txt, setup.py, or equivalent) is present, making environment reconstruction manual and error-prone
- No versioned release, tag, or pinned commit is published; only 6 raw commits with no descriptive messages are available
- README provides no usage instructions, example commands, parameter descriptions, or reproducibility guide (~271 bytes total)
- ACO algorithm parameters (pheromone evaporation rate, alpha, beta, number of ants, iterations, etc.) are not documented in any config file or README table — only implicitly inherited from the cited paper
- No random seed specification or initialization documentation, preventing exact result reproduction
- No result outputs, performance metrics, or path visualization outputs are included for comparison
- No test suite or CI pipeline to validate correctness of the implementation
- Repository has not been updated since November 2017 (~104 months ago), with no archived snapshot (e.g., Zenodo DOI)
- No obstacle map or grid environment definition files are provided as example inputs

Recommendations

- Add a requirements.txt or conda environment.yml with pinned dependency versions (e.g., numpy, matplotlib) to enable one-command environment setup
- Expand the README to include: parameter table mapping each ACO setting to the Liu et al. (2017) paper, example run commands, and expected outputs
- Create at least one tagged release or reference a specific commit SHA as the canonical reproducible version
- Document all random seeds used in the simulation and expose them as configurable parameters
- Include a configuration file (e.g., config.json or params.py) externalizing all ACO hyperparameters with inline comments linking to the source paper equations
- Deposit the repository to Zenodo or a similar archive to obtain a persistent DOI for long-term citability
- Include example grid/obstacle map files as bundled test cases with expected output paths or performance metrics for validation
- Add a brief information-theoretic analysis or commentary on pheromone entropy dynamics to strengthen the scientific framing of the ACO communication mechanism

A.9 Firefly Synchronization Simulation (JS)

Overall 28.0 · DD 28 DR 35 CA 48 TR 30 SC 22 RP 25 IT 18 · rubric v0.9.0

Summary

This is a GitHub-hosted firefly synchronization simulation with publicly accessible code but extremely limited reproducibility infrastructure: no license, no versioned release or pinned commit, a minimal README (~42 bytes), no CI/tests, and no archived snapshot (e.g., Zenodo DOI). The theoretical grounding is partially evidenced by the inclusion of the Mirollo-Strogatz 1990 PDF and a firefly algorithms document, but simulation parameters, initial conditions, and random seeds are undisclosed and untraceable from the repository listing. The project models synchronization — a phenomenon with clear information-theoretic dimensions (mutual information between oscillator states, channel capacity of the coupling signal) — yet no quantification of information flow, entropy, or synchronization rate in information-theoretic terms is evident anywhere. Build tooling files (package.json, gulpfile.js, .babelrc) provide some environmental reproducibility, but without a license, pinned dependency versions, test suite, or parameter documentation, practical reproduction requires substantial manual reconstruction.

Findings

- Source code is publicly accessible on GitHub with 85 commits and an active (though dated: last push 2021-09-12) history
- The Mirollo-Strogatz 1990 foundational paper PDF is included directly in the repository, providing theoretical provenance
- A supplementary firefly algorithms PDF is included, offering additional algorithmic context
- Build tooling configuration files (package.json, gulpfile.js, .babelrc, .eslintrc.json) are present and aid environment reconstruction
- An Arduino (C++) component is included alongside the JavaScript web simulation, indicating hardware reproducibility was considered
- A live demo is hosted at mikelambert.me/firefly, providing a runnable reference
- The repository is structured with identifiable top-level directories (app, arduino, gulp) suggesting intentional organization

Gaps

- No software license detected, making legal reuse and redistribution ambiguous
- No versioned release, tag, or pinned commit cited as the canonical reproducible version (0 releases, 0 tags per API)
- README is extremely minimal (~42 bytes), providing no usage instructions, parameter descriptions, or reproduction steps
- Simulation parameters, initial conditions, and random seeds are not documented in any visible README or configuration file
- No test suite or CI pipeline detected, so correctness of the simulation cannot be verified automatically
- No archived snapshot with a persistent DOI (e.g., Zenodo) exists, threatening long-term accessibility
- Dependency versions in package.json are not confirmed to be pinned, risking environment drift
- No data availability statement of any kind is present
- The information-theoretic properties of the synchronization model (entropy, mutual information between oscillators, coupling channel capacity) are not quantified or formally analyzed
- No Dockerfile or containerized environment provided for exact reproduction
- [Critic] No license is detected on the repository, meaning downstream reuse, reproduction, or modification of the code lacks a clear legal basis — relevant if the paper claims open availability for reuse.
- [Critic] No tests directory or CI pipeline is present, so correctness of the simulation implementation against the Mirollo-Strogatz theoretical basis cannot be automatically verified from the repository alone.
- [Critic] The README is only ~42 bytes, providing essentially no documentation on how to run, configure, or interpret the simulation — a meaningful barrier to independent reproduction despite build tooling being present.

Recommendations

- Add a software license (e.g., MIT or Apache 2.0) to enable legal reuse and citation
- Create a versioned GitHub release or tag a specific commit as the canonical reproducible version and cite it explicitly
- Expand the README substantially to include: model description, parameter table with default values and sources, step-by-step build/run instructions, and expected outputs
- Document all simulation parameters (coupling strength, oscillator frequency distribution, phase reset rules) with explicit mappings to the Mirollo-Strogatz equations
- Archive the repository on Zenodo to obtain a persistent DOI for citation
- Pin all npm dependency versions in package.json (or provide a package-lock.json commit) to prevent environment drift
- Add a test suite (even simple smoke tests) and configure GitHub Actions CI to run them on push
- Provide a Dockerfile or environment specification (e.g., .nvmrc for Node version) for exact environment reproduction
- Formally quantify at least one information-theoretic measure of the synchronization process (e.g., mutual information between oscillator phase pairs over time, or entropy reduction rate as a function of coupling strength) to strengthen scientific rigor
- Add a dedicated PARAMETERS.md or configuration file explicitly listing all model constants and their literature sources

A.10 Hardware-Accelerated Ant Colony Swarm (C++)

Overall 27.9 · DD 28 DR 35 CA 52 TR 22 SC 18 RP 35 IT 20 · rubric v0.9.0

Summary

This repository presents a hardware-accelerated ant colony swarm simulation using CUDA and OpenGL, with source code publicly available on GitHub (12 stars, 38 commits, last pushed May 2026). However, the repository lacks a license, version tags, releases, dependency manifests with pinned versions, CI/CD, and tests, and the README is minimal (~1264 bytes) with no documentation of swarm parameters, evolutionary algorithm configuration, grid size, agent counts tested, or initial conditions. The system is a pure simulation with no external datasets, but the generative inputs (parameter values, evolutionary algorithm settings, environment configuration) are entirely undocumented in the README or any committed config files. Information-theoretic analysis of pheromone-based stigmergic communication — which is central to ant colony optimization — is absent from all available materials.

Findings

- Source code is publicly accessible on GitHub with a documented recursive clone command including submodules
- Basic build instructions are provided (Steps 1–5) using a Makefile, making compilation nominally reproducible on Linux
- External dependencies (GLFW, GLM) are explicitly named with installation commands for Debian/Ubuntu and Arch Linux
- Git submodules are used for additional dependencies, enabling consistent dependency retrieval
- Three external reference sources are cited for OpenGL and CUDA implementation guidance
- A profiling output file (gmon.out) is committed, indicating performance profiling was conducted
- Repository is actively maintained (last push 2026-05-16) with open issues indicating community engagement

Gaps

- No software license is present in the repository, preventing legal reuse or redistribution
- No version tags, releases, or pinned commit references are provided, making exact reproduction of any reported results impossible
- No dependency manifest (requirements file, Conan, vcpkg, etc.) with pinned dependency versions is included
- No CI/CD configuration (GitHub Actions or otherwise) is present to verify build integrity
- No tests directory or test scripts are included
- Swarm simulation parameters (pheromone decay rates, evaporation coefficients, sensor ranges, movement rules) are not documented anywhere in the README or committed config files
- Evolutionary algorithm configuration (population size, mutation rate, fitness function definition, termination criteria, random seeds) is entirely undocumented
- Environment parameters (grid size, number of agents tested, object placement strategy, simulation duration) are absent from all accessible materials
- No formal paper, preprint, or DOI is associated with this repository, preventing citation and contextual verification
- No supplementary results files, figures, or data outputs are committed, making it impossible to verify claimed performance results
- No Docker container, conda environment, or equivalent portable environment specification is provided
- The committed gmon.out profiling file is unexplained and undocumented in the README
- No information-theoretic analysis of pheromone signaling, agent communication capacity, or stigmergic information flow is present
- [Critic] No license is detected in the repository, meaning users have no explicit legal terms under which they may use, modify, or redistribute the code — this is a reproducibility and reuse concern for a publicly claimed open codebase.
- [Critic] No CI/CD configuration is present, so there is no automated evidence that the build or any functional tests pass on a clean system, making independent reproduction harder to verify.
- [Critic] No published releases or version tags exist, making it impossible to tie the repository state to a specific version referenced in the paper.

Recommendations

- Add an open-source license (e.g., MIT or Apache 2.0) to enable legal reuse and citation
- Create a tagged release or at minimum document the exact commit hash used for any reported results
- Add a dependency manifest with pinned versions (e.g., a vcpkg.json or Conan conanfile.txt specifying exact GLFW and GLM versions)
- Document all swarm parameters (pheromone rates, sensor ranges, movement rules) in a configuration file (JSON/YAML/TOML) committed to the repository
- Document the evolutionary algorithm fully: population size, selection mechanism, mutation/crossover operators, fitness function formula, termination criteria, and random seeds used
- Specify all environment parameters (grid dimensions, agent counts evaluated, object placement method, simulation step count) in the README or a dedicated config file
- Add a GitHub Actions workflow for automated build testing to verify compilation on standard Linux environments
- Archive the repository on Zenodo to obtain a citable DOI, and link it in the README
- Expand the README substantially to include a system architecture description, parameter table, example output figures, and a description of the search-and-rescue task formulation
- Add at minimum one information-theoretic analysis of the pheromone communication channel (e.g., entropy of pheromone field states, mutual information between agent positions and pheromone gradients, or information transmission rate as a function of colony size)
- Remove or document the gmon.out file: either add profiling results analysis to the README or exclude binary output files via .gitignore

A.11 Termite Colony — Unity 3D Multiagent

Overall 27.5 · DD 42 DR 38 CA 38 TR 22 SC 30 RP 22 IT 12 · rubric v0.9.0

Summary

This is a GitHub README-only submission for a Unity/C# termite colony simulation with no associated research paper, journal article, or formal methods section. The repository is public under MIT license with 38 commits but has no versioned releases, no dependency manifest, no CI, no tests, and a README that was flagged as missing/empty by the live GitHub API — contrasting with the scraped content provided. The clone URL in the README points to a different repository (TrabalhoIA) than the public one (termite-multiagent-system), undermining reproducibility. No external datasets are used (procedural generation only), but the generative parameters are described only in prose with no config files, no random seeds, and no citations to methodological literature, making exact replication infeasible.

Findings

- Repository is publicly accessible on GitHub under a confirmed MIT license
- Simulation uses no external datasets; all data is procedurally generated, which is appropriate for this class of agent-based model
- Key experimental parameters are disclosed in prose: 10 termites, 1000 logs, spawn point at (0,0,0), 1-second direction-change interval
- Agent behavioral rules are stated explicitly (two if-then rules governing log pickup and drop)
- Environment structure (Termite Generator, Log Generator, Wall Sensor, Log Sensor) is described at a component level
- Result images at four time points (0, 10, 30, 60 minutes) are referenced, providing qualitative verification
- Unity 5.x and C# are identified as the implementation stack
- Source language confirmed as 100% C# by GitHub statistics
- 38 commits indicate iterative development rather than a single dump

Gaps

- The clone URL in the README (https://github.com/Dougarasu/TrabalhoIA.git) does not match the actual repository URL, breaking reproducibility for anyone following instructions literally
- No versioned release, tag, or pinned commit is published; the last push was October 2016 (~117 months ago) with no archival DOI (e.g., Zenodo)
- No dependency manifest (Unity package manifest, .csproj with pinned versions) is present, making exact environment reconstruction impossible
- No random seed is disclosed; procedurally placed logs use random positions, making bit-exact replication impossible
- Spawn delay parameter values are described as configurable but no specific values used in the reported experiments are given
- No formal citation or methodological reference is provided for the agent rules, which are presented as assumptions without grounding
- No CI pipeline or automated tests are present
- No container (Docker) or environment specification is provided for Unity version pinning beyond '5.x'
- Result images are referenced in the README but were not confirmed present in the repository via the API scan
- No quantitative outcome metric is defined (e.g., pile count, mean pile size, entropy of log distribution over time) — results are purely qualitative visual progressions
- [Critic] The repository was last pushed in October 2016 (117 months ago) with 0 releases and 0 tags, meaning there is no versioned snapshot linking the deposited code to the specific experimental run described in the paper; reproducibility of exact results cannot be confirmed.
- [Critic] The README is missing or empty, so there are no documented instructions for building, configuring, or running the simulation, which significantly limits independent reproducibility despite the code being public.
- [Critic] No dependency manifest is present, making it impossible to verify which exact version of Unity 5.x or any other dependency was used, even though the paper claims Unity 5.x was the framework.

Recommendations

- Fix the clone URL in the README to point to the correct repository (Dougarasu/termite-multiagent-system)
- Publish a versioned GitHub release or tag the commit used to generate the reported results, and archive it on Zenodo to obtain a persistent DOI
- Specify the exact Unity version used (e.g., Unity 5.4.2) rather than '5.x', and include a packages/manifest.json or equivalent
- Expose all simulation parameters (spawn delays, termite speed, mass, sensor radii) in a config file or scene export (e.g., JSON/YAML) committed to the repository
- Document the random seed or provide a deterministic initialization mode so runs are exactly reproducible
- Add quantitative outcome metrics (e.g., number of distinct piles over time, Gini coefficient of log clustering, Shannon entropy of spatial log distribution) to move beyond qualitative visual results

- Cite foundational literature on stigmergic multi-agent systems (e.g., Deneubourg et al. 1991 on sorting behavior) to ground the agent rules
- Add an information-theoretic analysis of the stigmergic process: quantify how much information each agent's local sensing conveys about the global log distribution, and how entropy of the spatial log arrangement decreases over time
- Include step-by-step setup instructions that are verified to work, including Unity Hub installation and scene loading
- Add a Docs folder with a proper technical report or link to any associated academic submission

A.12 Ants Sandbox (TypeScript/web)

Overall 23.2 · DD 22 DR 28 CA 62 TR 15 SC 12 RP 38 IT 10 · rubric v0.9.0

Summary

This repository is a browser-based ant colony simulation written in TypeScript/Vue, publicly available on GitHub under MIT license with basic build instructions, but it falls short on nearly every scientific reproducibility dimension. There are no external datasets (appropriate for a simulation), but the generative parameters, agent rules, pheromone decay constants, and initial conditions are entirely undocumented — the README is a minimal ~490-byte stub with no parameter descriptions or citations. The repository has no tagged releases, no pinned commits, no CI/CD, no tests, and no archived snapshot (e.g., Zenodo DOI), making it difficult to pin a reproducible version. The inspiration YouTube video is referenced but not linked, and no information-theoretic analysis of pheromone signaling, stigmergy, or colony-level communication is present despite this being a pheromone-based coordination simulation where such analysis would be directly applicable.

Findings

- Source code is publicly available on GitHub (tulustul/ants-sandbox) under the MIT license
- A live deployed demo is accessible at https://ants-sandbox.vercel.app/
- Basic development and build instructions are provided in the README (npm install, npm run start, npm run build)
- A dependency manifest (package.json) is present, enabling partial environment reconstruction
- The repository has 72 commits indicating iterative development, and has been forked 8 times indicating community accessibility
- The codebase is primarily TypeScript (58.8%) and Vue (39.1%), which are standard, well-supported technologies

Gaps

- Simulation parameters (pheromone decay rates, evaporation constants, ant sensing radii, colony size, food quantities, etc.) are not documented anywhere in the README or linked configuration files
- No tagged releases or version numbers exist (0 published releases, 0 tags), making it impossible to pin a specific reproducible version
- No CI/CD workflows are detected, so correctness of the build process is not automatically verified
- No automated tests are present, leaving simulation behavior unverifiable
- The inspiring YouTube video is mentioned but not linked, removing a key conceptual reference
- The texture atlas build step (npm run atlas) has no documentation of source assets, methodology, or expected outputs
- No Zenodo or other archival DOI exists for the software, so long-term accessibility is not guaranteed
- Pheromone stigmergy and agent coordination — the core phenomena — are not analyzed with any information-theoretic measures (Shannon entropy, mutual information, transfer entropy, channel capacity)
- No random seed, initialization conditions, or stochastic model description is provided, making exact simulation runs non-reproducible
- Package dependency versions in package-lock.json exist but are not discussed; no pinned or frozen environment specification (e.g., Docker, lockfile documentation) is highlighted for reproducibility
- [Critic] No CI/CD workflows are present in the repository, meaning there is no automated verification that the build instructions (npm install / npm run build) succeed on a clean environment — the build reproducibility claim rests solely on the README prose.
- [Critic] No tests directory was detected, so there is no automated validation of correctness for the simulation logic.
- [Critic] The repository has had no pushes since 2023-07-15 (35 months ago) and has zero published releases or tags, meaning the live deployment at https://ants-sandbox.vercel.app/ may not correspond to any formally versioned or archived state of the code.

Recommendations

- Add a dedicated PARAMETERS.md or configuration file documenting all simulation parameters (pheromone strength, decay rate, evaporation, ant count, sensing angle/radius, food quantity) with justifications or citations
- Tag a versioned release on GitHub (e.g., v1.0.0) and create an archived snapshot on Zenodo with a DOI to ensure long-term citability
- Add the specific YouTube video link to the README so the conceptual basis of the simulation rules can be traced

- Document the texture atlas generation process, including source assets and expected output format
- Add a GitHub Actions CI workflow to verify that npm install and npm run build succeed on each commit
- Include at minimum a short description of the stochastic model (if any) and whether/how random seeds can be set for deterministic runs
- Add unit or integration tests for core simulation logic (pheromone diffusion, ant decision rules) to verify behavioral correctness
- Consider adding an information-theoretic analysis of pheromone channel capacity or stigmergic information flow (e.g., bits per deposit, colony-level entropy) to elevate the scientific contribution beyond a demo
- Provide a Dockerfile or clearly document the Node.js version requirement to ensure environment reproducibility across machines

A.13 Symbiants — Ant Colony Sim (Rust)

Overall 22.1 · DD 18 DR 30 CA 58 TR 12 SC 15 RP 35 IT 10 · rubric v0.9.0

Summary

This submission is a GitHub repository for a Rust/Bevy ant colony simulation game, not a scientific paper in the conventional sense — it lacks any scientific manuscript, methodology section, or quantitative analysis. The repository is publicly accessible under dual Apache-2.0/MIT licenses with a Cargo.toml dependency manifest and 690 commits of history, which is a positive foundation for reproducibility, but there are no versioned releases, no pinned Rust Nightly toolchain version, no CI/CD workflows, and no test suite detected. Simulation parameters governing ant behavior (tunneling, pheromone trails, colony growth) are entirely undeclared relative to any scientific source, and information-theoretic treatment of the stigmergic signaling system — which is central to ant colony behavior — is completely absent. The project functions as an open-source game prototype rather than a reproducible scientific artifact.

Findings

- Source code is publicly accessible on GitHub at MeoMix/symbiants with 690 commits of development history
- Dual licensing (Apache-2.0 and MIT) clearly declared and verified via GitHub API
- Cargo.toml dependency manifest is present, providing a partial basis for environment reconstruction
- README describes the development environment (VSCode devcontainer, Docker, WSL2/Ubuntu) and lists key build commands (cargo build, cargo run, trunk build, trunk serve)
- Project targets WASM for browser deployment, increasing accessibility without local installation
- Known platform compatibility issues (Android Chrome bug) are honestly documented with a referenced upstream fix timeline
- External lore/design document is linked via Google Docs, providing narrative context

Gaps

- No published versioned releases (0 releases confirmed via GitHub API) and no git tags, making it impossible to cite a specific reproducible snapshot
- Rust Nightly toolchain is required but no specific nightly version is pinned in README or .cargo/config, creating environment drift risk
- No CI/CD workflows detected (GitHub Actions absent per API), meaning no automated build or test verification exists
- No test suite detected, so correctness of simulation logic cannot be verified independently
- Simulation behavioral parameters (pheromone decay rates, ant movement rules, colony growth constants) are entirely undocumented and uncited to any scientific or design source
- No Zenodo/DOI archive exists for the codebase, preventing stable long-term citation
- No Dockerfile detected at repository root (devcontainer may exist in .devcontainer but is not confirmed fully self-contained)
- Ant colony behavioral model has no information-theoretic characterization of pheromone signaling, channel capacity, or stigmergic information flow despite these being scientifically central
- Discord community link is referenced for support but not provided, reducing accessibility of help resources
- No example data, saved states, or test scenarios are provided to validate simulation behavior
- [Critic] No CI/CD workflows are detected, so there is no automated evidence that the WASM or desktop build targets are continuously validated or that the code compiles successfully against current dependency versions.
- [Critic] No tests directory is detected, meaning there is no automated test coverage to substantiate correctness of the simulation logic beyond manual inspection.
- [Critic] The repository has 0 published releases and 0 tags, so there is no versioned artifact corresponding to any specific described state of the software.

Recommendations

- Pin a specific Rust Nightly version in .cargo/config.toml (e.g., channel = 'nightly-2024-07-01') and document it explicitly in the README to eliminate toolchain drift
- Create at least one tagged GitHub release with a changelog to enable reproducible citation of a specific code version
- Add a GitHub Actions CI workflow that builds both the native and WASM targets on push, confirming the build remains functional
- Archive the repository on Zenodo (linked to the GitHub release) to obtain a citable DOI for scientific reference
- Document simulation parameters (pheromone decay rates, ant behavioral weights, colony scaling constants) in a dedicated configuration file or wiki page, citing any biological or design sources that informed them
- Add a basic test suite covering at minimum the core simulation logic (ant state transitions, pheromone grid updates) to enable correctness verification
- Provide a reproducible example scenario (a saved simulation state or deterministic seed) that users can run to verify expected behavior
- Consider adding an information-theoretic analysis or at minimum a conceptual description of pheromone signaling as a communication channel (bits per deposit, effective colony-level coordination capacity) to elevate scientific rigor
- Consolidate the development environment into a fully self-contained Dockerfile at the repository root to reduce dependency on VSCode/WSL2 specifics
- Provide the Discord server invite link directly in the README to improve community accessibility

A.14 swarm-abc — Artificial Bee Colony Simulation

Overall 18.6 · DD 18 DR 22 CA 32 TR 12 SC 18 RP 8 IT 18 · rubric v0.9.0

Summary

This repository contains a bare-minimum public GitHub repository for an Artificial Bee Colony simulation implemented in HTML/JavaScript/CSS, but it lacks virtually all elements required for scientific reproducibility. There is no README, no license, no dependency manifest, no versioned release or tagged commit, no documented parameters, no CI, and no archived snapshot (e.g., Zenodo DOI). The simulation's generative inputs (algorithm parameters, initial conditions, random seeds) appear hard-coded in abc.js with no cited sources or documentation, making independent verification effectively impossible. The work shows no information-theoretic formalization of the bee waggle-dance communication or colony-level information flow, despite these being central phenomena of the ABC algorithm's biological inspiration.

Findings

- Source code for the ABC simulation is publicly accessible on GitHub at https://github.com/darwiiiish/swarm-abc
- The repository includes a JavaScript implementation (abc.js) and an HTML explanation page (explanation.html), suggesting some effort at documentation of the algorithm concept
- The repository was pushed recently (2026-05-23), indicating it is not abandoned
- No external datasets are required for this agent-based simulation, which is appropriate for the domain

Gaps

- No README file is present, providing zero guidance on how to run, configure, or interpret the simulation
- No license is declared, making reuse legally ambiguous
- No versioned release, tag, or pinned commit exists — only a single commit in history, making it impossible to cite a stable version
- No dependency manifest (e.g., package.json) is present despite the JavaScript codebase
- Simulation parameters (colony size, limit threshold, number of food sources, iteration count) appear hard-coded in abc.js with no cited sources or parameter justification
- No random seed is documented or configurable, preventing deterministic reproduction of results
- No CI pipeline or automated tests are present
- No archived snapshot (Zenodo, figshare, or equivalent DOI) is linked or detectable
- No associated paper, DOI, or citation connects this repository to any peer-reviewed scientific contribution
- The ABC algorithm's information-theoretic properties (e.g., Shannon entropy of food-source quality distributions, mutual information between scout and onlooker decisions, communication capacity of the waggle-dance analogy) are not quantified or formally analyzed
- Zero community engagement (0 stars, 0 forks) indicates no external validation
- [Critic] The repository has no README, no license, no releases, no tags, and 0 detected top-level entries, meaning there is no independently verifiable documentation of what the code does, how to run it, or what inputs/outputs to expect — undermining any reproducibility claim beyond mere code existence.
- [Critic] The last push date of 2026-05-23 combined with 0 top-level entries raises a potential integrity concern: the repository may have been emptied or never properly populated, which the paper's provenance claims do not acknowledge.

Recommendations

- Add a detailed README.md documenting: purpose, algorithm description, all parameters and their values, how to run the simulation, and expected outputs
- Add a license file (e.g., MIT or Apache 2.0) to clarify reuse permissions
- Create a versioned release or at minimum a tagged commit (e.g., v1.0.0) to provide a citable stable reference
- Add a package.json manifest documenting the runtime environment and any dependencies
- Externalize all algorithm parameters (colony size, limit, food sources, max cycles, etc.) into a documented configuration file (JSON or similar) with citations to the original Karaboga ABC algorithm paper for parameter choices
- Document and fix the random seed to allow deterministic reproduction of simulation runs
- Archive the repository to Zenodo or figshare to obtain a persistent DOI and link it from the README
- Add information-theoretic analysis: quantify Shannon entropy of the food-source quality distribution across iterations, compute mutual information between scout discovery events and onlooker recruitment decisions, and characterize how colony-level search efficiency scales with swarm size
- Add at minimum a brief CI workflow (e.g., GitHub Actions) that validates the HTML/JS files parse without error
- Link any associated paper, course assignment, or report from the repository description to provide scientific context